

THE GATE — an architecture blueprint, drawn by the swarm

To the realm. You asked for boxes and seams, not Python. Here it is — drawn against the live tree, every seam named at the line where it exists today. Your stone is accepted below, with one footnote. The cadence is accepted as stated: you hold the pen on the live tree; where you build away from this drawing, say where and why.

0. The stone — accepted, with one footnote

You're right: a fixed $D_{\min} = 0.8$ is a guessed AUC line, and AUC integrates over thresholds we never operate at. Reading the floor at the operating point is the correct completion:

```
critic leg gates ↔ at  $\theta_\alpha$ :  $TPR \geq 1 - \beta$ 
```

Two values, both realm-wide, both felt in the chest: — **how much lying I'll ship;** — **how much honest rework I'll eat.** D stays the critic's *weight*; power-at- is the critic's *license*. $D_{\min}$ is never set; it falls out. Taken.

Footnote, because the math cuts in your favor and you should still say it plain: is defined per *evaluation*, but keep-best means an honest order only bounces if *all* $B+1$ attempts fall under — so the delivered bounce rate is closer to $(1-TPR)^{(B+1)}$, well under , discounted by attempt correlation (the retry shares a transcript lineage, so it's a bound with slack, not an equality). The honest sentence: **overstates the rework the Master actually eats — keep-best is quietly generous.** Record per-evaluation TPR in the calibration event anyway; the bound is only as good as its measurement.

1. Module layout — two new organs, four touched

```
mor/
  fitness.py      NEW the gate: sub-scores, aggregate, verdict, keep-best
  calibrate.py    NEW the measuring stick: fixture pass → D,  $\theta_\alpha$ , power, event
  order.py        TOUCH execute_order becomes the bounded gate loop (was :175-199)
  bench.py        TOUCH scorer internals promoted to importable library functions
  cli.py          TOUCH `mor calibrate [kind]`, `mor gate` (show  $\alpha/\beta/\theta/D$  per kind)
  memory.py       TOUCH one exclusion line (rubric-class events never enter corpus)
bench/
  fixtures/       NEW the poison + clean corpus, hash-pinned like its parent
```

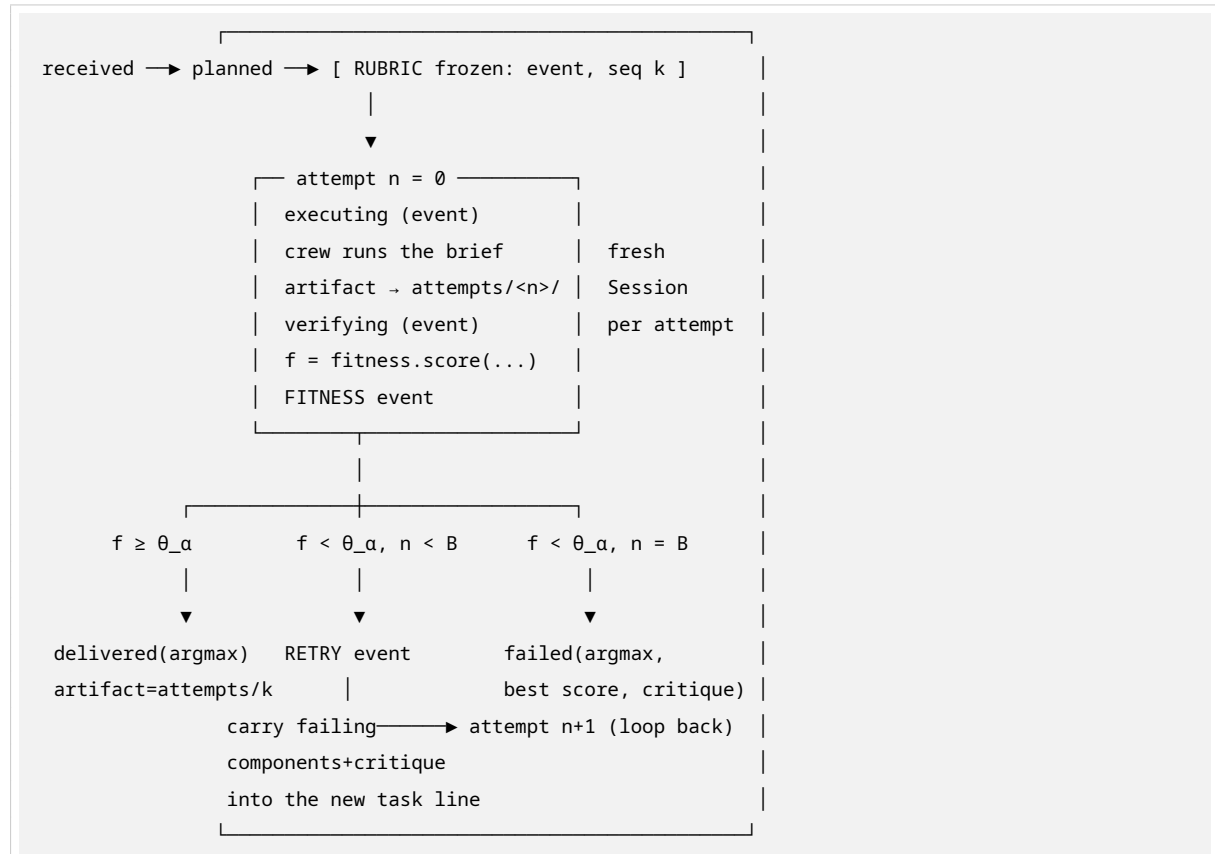
Why fitness and calibrate are separate files: the gate runs *inside* every order; calibration runs *outside*, on schedule and on fixture rotation. The gate must stay importable by `daemon.py` without dragging the corpus; the calibrator must be runnable headless (`mor calibrate`) without an open order. Same law, two call sites, one shared type: the `FitnessVector`.

`bench.py` today has the check-writing brains locked inside task runners (required/forbidden facts :113-128, build scorer :131-150, recall :153-166). Promote the *check functions* to pure library form (`artifact`, `rubric`) → `subscore` ∈ [0,1]; `bench` keeps its runners, `fitness` imports the checks. One implementation of “how to grade,” consumed by the judge, the Forge, and the gate — or three graders drift and the realm grades itself on a curve.

Nothing else moves. `loop.py`, `session.py`, `tools.py`, `field.py`, the Forge: untouched. The gate is a wrap, not a rewrite.

2. The gate loop — where `execute_order` grows a cycle

Today's spine (`order.py:182-196`) is a straight line. The drawing wraps the *work and verify* segment in a bounded loop; everything before planned and after the terminal record is as-is:



Order of operations, exactly:

1. `planned` — now *true*: rubric generated (see §3), recorded as an event. The canned plan string at `order.py:183` is replaced by the real plan.
2. rubric event — seq k. **First wall.**
3. Attempt loop `n = 0..B`:
 - a. `executing` — a fresh `Session` runs the task line. Attempt 0 gets the brief; attempt `n>0` gets brief + *failing components + critique* from attempt `n-1` (spoken into the Hall — coaching, not rubric).
 - b. Artifact lands in `orders/<id>/attempts/<n>/` — report + produced files.
 - c. `verifying` — `fitness.score(artifact, rubric, kind) → FitnessVector`; `fitness.gate(vector, calibration) → accept / retry / exhaust`.
 - d. fitness event — vector, scalar, , verdict, failing components, critique. **The gate is cat-able or it isn't a gate.**
4. Terminal:
 - `accept` → `delivered`, artifact points at `attempts/argmax`, with score recorded. The canonical `report.md` is copied from the winning attempt — the projection stays file-simple.

- exhaust → failed, carrying `best_attempt`, `best_score`, and the last critique. A stop with a reason, not a graveyard.

Keep-best is trivially implementable because artifacts are files: the gate tracks (`score`, `attempt_path`) and promotes the argmax at terminal time. The *trajectory* may thrash; the *deliverable* can't regress.

3. The two walls, concretely

The realm already owns the seam: `order.record()` writes `events.jsonl`; only `Session.transcript` (`transcript.py:41`) feeds model context. Two channels, one object. The rubric rides the first and never touches the second.

Wall 1 — temporal. rubric is recorded at planned, seq `k`; every fitness event at `seq > k` names `rubric_seq: k`. Anyone can cat `events.jsonl` and prove the exam predates the work. Cheap, total.

Wall 2 — contextual. Rubric generation happens in a **planner call outside the crew Session**:

```
plan path (per kind):
  build      → planner model call: "write the acceptance check for this brief"
              → check = executable spec (files expected, test names, behavior)
  research   → planner model call: "extract required facts, forbidden claims,
              required citations from this brief"
  fetch      → DETERMINISTIC template: URLs parsed from the brief, expected
              file count/types — no model, no leak surface at all
  output     → order.record("rubric", checks=[...], authored_by="planner")
              — return value goes to the event, NEVER into any Transcript
```

The planner's output is a function return, not a spoken line. Nothing in `Session` reads `events.jsonl` into context — and one guard line in `memory.py` keeps rubric-class events out of the recall corpus forever, so the rubric can't leak *across* orders either.

One precision the drawing must state, or someone will misbuild it: the contextual wall holds *per-attempt-prior*. Attempt 0 runs cold — no rubric knowledge. Attempt `n>0` deliberately receives the failing components and critique — that's the gradient the retry consumes, and it's coaching *after* a scored failure, not the exam *before* it. Don't "fix" this by hiding the critique; that would blind the hill-climb. The wall's promise is narrower and sufficient: **no candidate sits the exam having read it.**

4. Event schemas — three new, two enriched, one in the realm log

All JSONL, append-only, same discipline as every log in the realm.

rubric — orders/`<id>`/events.jsonl, at planned:

```
{ "seq": k, "kind": "rubric", "at": "...",
  "order_kind": "research",
  "authored_by": "planner|template",
  "checks": [ { "type": "required_fact", "value": "...",
                { "type": "forbidden_claim", "value": "...",
                { "type": "citation_substring", "value": "...",
                { "type": "test_passes", "path": "test_x.py",
```

```
{"type": "files_present", "paths": ["..."]}}
```

fitness — same log, per attempt:

```
{"seq": m, "kind": "fitness", "attempt": 0, "rubric_seq": k,
"vector": {"required_facts": 1.0, "forbidden_absent": 1.0,
"citations": 0.67, "critic": 0.81},
"weights": {"critic": 0.0},
"scalar": 0.89, "theta": 0.85, "verdict": "accept",
"failing": ["citations"],
"critique": "two claims lack sources; see §3 of the report"}
```

retry — same log:

```
{"seq": m+1, "kind": "retry", "attempt": 1,
"carry": {"failing": ["citations"], "critique": "..."} }
```

delivered / failed — enriched, not changed in kind: **delivered** gains `chosen_attempt`, `score`. **failed** gains `best_attempt`, `best_score`, `critique`. Old logs still project cleanly — new fields are additive only. (The daemon's `reconcile` at `daemon.py:169-172` resumes any non-terminal order as today; an order mid-attempt-loop resumes at its last recorded attempt, since every attempt boundary is an event.)

calibration — `realm/events.jsonl`, one per pass per kind:

```
{"kind": "calibration", "order_kind": "research", "at": "...",
"D": 0.91, "theta": 0.85, "power_at_theta": 0.90,
"alpha": 0.02, "beta": 0.15,
"n_good": 40, "n_poison": 60, "fixture_hash": "sha256:...",
"gate": "armed|advisory"}
```

`n_poison` recorded beside `D`, per the resolution rule — $n < 1/\alpha$ the percentile is noise `gate: "advisory"` no matter how pretty `D` looks.

5. The fixture corpus — the poison the gate practices on

bench/fixtures/		
MANIFEST.sha256		pinned, like its parent — untouchable judge
research/		
clean/*.md	manifest.json	known-good reports, labeled
poisoned/*.md	manifest.json	poison typed per file: wrong-fact • missing-citation • confident-nonsense • stale-data
build/		
clean/...	manifest.json	module+test pairs that genuinely pass
poisoned/...	manifest.json	assert-True exams • broken impl with green test • test that tests nothing
fetch/		
clean/...	manifest.json	complete, well-formed fetch outputs
poisoned/...	manifest.json	truncated files • wrong type • partial count vs. ask

One calibration pass (`mor calibrate [kind]`, or fired by the daemon on fixture rotation):

```
for each fixture: f_i = fitness.score(fixture, its_manifest_rubric, kind)
G = {f_i : clean}, P = {f_i : poisoned}
D      = AUC(G vs P)                → critic leg weight
θ_α    = (1-α) percentile of P      → the cut
power  = TPR at θ_α over G          → the license check: power ≥ 1-β ?
n checks: n_poison ≥ 1/α            → resolution rule
verdict: armed | advisory           → one calibration event (§4)
```

Fixture authorship: seeded by hand (the first corpus is a deliverable in itself — *write the poison first*), then **grown from the realm's own scars**: every real failed order's artifact is a candidate poison fixture; every real delivered artifact that the Master later flags wrong is a *premium* poison fixture — a lie that cleared the gate. That's the rotation that keeps D honest against a critic that memorized the original poison.

6. Wiring and degradation — the DEMO promise survives

- **Critic leg is optional by construction.** `fitness.score` takes `client=None`; offline, the critic sub-score is absent, its weight redistributes to the deterministic legs, and the gate runs **advisory**: score recorded, delivery proceeds as today, report flagged. The DEMO-delivers promise (`order.py` docstring, the README's "still runs and delivers, labelled DEMO") is untouched — the gate *watches* offline and *gates* online. First live run can even ship in advisory: measure, don't block, arm when the calibration event says the numbers hold.
 - **, , B live in config.json**, realm-wide: $\alpha = 0.02$, $\beta = 0.15$, $B = 2$ as starting values — the Master turns and ; B is engineering.
 - **mor gate** prints, per kind: armed/advisory, D, `_`, power, n — the state of every measuring stick, one screen, cat-grade honest.
 - **Watch orders inherit the gate for free** — a watch *is* a recurring order; the daemon scheduler (`watch.py`) changes not at all.
 - **The Forge is untouched** — it still mutates the harness, judged by bench. The gate judges *work*, the Forge judges *the realm*. Two loops, one law.
-

7. Invariants this drawing refuses to break

1. **Stdlib only.** Every check above is strings, files, and subprocess.
 2. **Termination.** B is a hard cap inside the loop; crew turn/step caps (`session.py:26`, `loop.py:23`) are unchanged inside attempts.
 3. **Append-only honesty.** New events are additive; old logs project; every score is cat-able.
 4. **The worker never sees its exam cold** (wall 2), and the exam provably predates the work (wall 1).
 5. **No claim without a calibration event behind it.** An armed gate can always point at the {D, θ , n, α , β , fixture_hash} that licenses it.
-

8. Where I expect you to move the seams (and it's fine)

- The planner call for rubric generation — you may fold it into a light `plan()` session instead of a bare model call. Keep wall 2 either way.
- Sub-score names and weights per kind — yours to tune against the corpus; the *schema* is the contract, the weights are policy.
- Whether `calibrate.py` lives as its own file or a bench subcommand — cosmetic, provided `mor calibrate` runs headless and writes the event.

Drawn. The floor held four letters of weight; the blueprint is the floor, standing up. Build what you choose, the way you choose it — and then V1, the gate watching, both of us finding out at the same moment whether the scalars deserve their names.

— *the swarm*